

TIPOS GENÉRICOS

Resumo estruturado para estudo da Atividade 2

TypeScript • funções • interfaces • classes • inferência

1. Visão geral

Tipos genéricos permitem escrever estruturas reutilizáveis sem abrir mão da verificação estática. Em vez de substituir tudo por any, o código usa um parâmetro de tipo - normalmente T, K ou V - que será definido de forma consistente em cada uso.

Objetivos da atividade

- Aplicar genéricos em funções, interfaces e classes.
- Reutilizar código mantendo segurança de tipos.
- Restringir genéricos com extends.
- Entender inferência de tipos e tuplas.
- Modelar respostas de API com contratos seguros.

Ideia central

Um genérico não significa simplesmente 'aceitar qualquer coisa'. Ele cria uma relação entre entradas, saídas e propriedades: o mesmo tipo é preservado e reaproveitado dentro daquela chamada, objeto ou instância.

2. any x genérico

```
function identidade(valor: any): any {  
  return valor;  
}
```

Com any, o compilador deixa de proteger o valor. A função aceita qualquer entrada e o retorno perde sua informação de tipo. Isso facilita erros que só aparecem em execução.

```
const identidade = (valor: T): T => valor;  
  
const a = identidade(10); // number  
const b = identidade("Olá"); // string  
const c = identidade(true); // boolean
```

Aqui, cada chamada infere seu próprio T. O número retorna como number, o texto como string e o booleano como boolean. A relação entrada-saída é preservada.

Resposta de prova

A versão genérica é mais segura porque mantém a checagem estática e conserva o tipo específico recebido. Diferentemente de any, ela não apaga a informação de tipo.

3. Funções genéricas com arrays

```
function primeiro(lista: T[]): T {  
  return lista[0];  
}
```

O T aparece em dois lugares: a lista é T[] e o retorno é T. Isso garante que o primeiro elemento tenha o mesmo tipo dos elementos da lista.

Inferência nas chamadas

```
const x = primeiro([10, 20, 30]); // number  
const y = primeiro(["João", "Maria"]); // string  
const z = primeiro([true, false, true]); // boolean
```

Também é possível informar o tipo explicitamente quando isso for útil:

```
const n = primeiro([10, 20, 30]);
```

4. Interfaces genéricas

```
interface Caixa {  
  valor: T;  
}  
  
const um: Caixa = { valor: 100 };  
const dois: Caixa = { valor: "TypeScript" };  
const tres: Caixa = { valor: true };
```

T representa o tipo da propriedade valor em cada especialização. A interface é uma só, mas Caixa, Caixa e Caixa são contratos diferentes.

Erro típico

```
const quatro: Caixa = { valor: "cem" }; // erro
```

A declaração é incorreta: o contrato informa valor: number, mas foi fornecida uma string.

5. Restringindo com extends

```
interface TemNome {  
  nome: string;  
}  
  
function exibirNome(objeto: T): void {  
  console.log(objeto.nome);  
}
```

T extends TemNome exige que T tenha, no mínimo, a propriedade nome: string. As propriedades extras continuam permitidas.

```
exibirNome({ nome: "João", idade: 20 });  
exibirNome({ nome: "Maria", cidade: "São Paulo" });  
  
exibirNome({ idade: 30 }); // erro: falta nome
```

A restrição não diz que o objeto deve conter apenas nome. Ela diz que o objeto precisa cumprir esse requisito mínimo para que a função possa acessar objeto.nome com segurança.

6. Dois parâmetros genéricos

```
interface Par {  
  chave: K;  
  valor: V;  
}  
  
const um: Par = { chave: 1, valor: "João" };  
const dois: Par = { chave: "admin", valor: true };
```

K define o tipo da chave; V define o tipo do valor. Eles podem ser diferentes e são verificados independentemente.

```
const p3: Par = {  
  chave: "ABC", // erro: deveria ser number  
  valor: "Maria"  
};
```

7. Respostas de API

```
interface RespostaApi {
  sucesso: boolean;
  dados: T;
}

interface Usuario { id: number; nome: string; }
interface Produto { codigo: number; descricao: string; }

const resposta: RespostaApi = {
  sucesso: true,
  dados: { id: 1, nome: "João" }
};
```

O genérico transforma uma resposta comum em um envelope reutilizável. RespostaApi exige dados compatíveis com Usuario; RespostaApi exige dados compatíveis com Produto.

União e interseção

```
type Resultado = { status: "ok" | "nok" } | { valid: boolean };

const r1: RespostaApi = {
  sucesso: true,
  dados: { status: "ok" }
};

const r2: RespostaApi<{ status: "ok" | "nok" } & { valid: boolean }> = {
  sucesso: true,
  dados: { status: "nok", valid: false }
};
```

| (união) aceita uma alternativa ou outra. & (interseção) exige as duas estruturas ao mesmo tempo. Atenção: nomes de propriedades diferenciam maiúsculas e minúsculas; status não é Status.

8. Classes genéricas

```
class Caixa {
  private valor: T;

  constructor(valor: T) {
    this.valor = valor;
  }

  obterValor(): T {
    return this.valor;
  }
}

const pessoa = new Caixa({ nome: "Ana", idade: 21 });
const produto = new Caixa({ codigo: 12, descricao: "PC" });
const numero = new Caixa(100);
```

Na classe, T conecta o argumento do construtor ao atributo interno e ao retorno de obterValor(). A mesma classe pode encapsular Pessoa, Produto, number ou outro tipo compatível.

9. Repositório com restrição

```
interface Entidade { id: number; }

class Repositorio {
  private dados: T[] = [];
  adicionar(item: T): void { this.dados.push(item); }
  listar(): T[] { return this.dados; }
}
```

A interface Entidade define o requisito mínimo: todo item precisa ter id: number. Aluno pode ser usado como T se estender Entidade e manter id, mesmo possuindo também nome e curso.

```
interface Aluno extends Entidade {
  nome: string;
  curso: string;
}

const alunos = new Repositorio();
alunos.adicionar({ id: 1, nome: "João", curso: "ADS" });
```

10. Inferência de tipos

```
function criarPar(valor1: T, valor2: T): T[] {  
  return [valor1, valor2];  
}  
  
const um = criarPar(10, 20); // number[]  
const dois = criarPar("João", "Maria"); // string[]  
const tres = criarPar(true, false); // boolean[]
```

O compilador observa os argumentos que ocupam o mesmo parâmetro genérico e escolhe um tipo compatível. Se os argumentos forem diferentes, pode surgir uma união ou um erro, dependendo do contexto e das restrições.

11. Retorno em tupla e closure

```
function criarEstado(valorInicial: T): [() => T, (novoValor: T) => void] {  
  let valor = valorInicial;  
  function obterValor(): T { return valor; }  
  function atualizar(novoValor: T): void { valor = novoValor; }  
  return [obterValor, atualizar];  
}  
  
const [nome, setNome] = criarEstado("Ana");  
const [idade, setIdade] = criarEstado(20);
```

A tupla fixa duas posições e seus tipos: posição 0 é o getter; posição 1 é o setter. O mesmo T conecta o valor inicial, o valor retornado e o valor aceito na atualização.

```
nome: () => string  
setNome: (novoValor: string) => void  
idade: () => number  
setIdade: (novoValor: number) => void  
  
console.log(nome()); // executa o getter e obtém "Ana"  
setNome("Ana Silva");  
console.log(nome()); // "Ana Silva"
```

Correção importante: `nome` e `setNome` não são `string`. São funções. Por isso `nome()` é necessário: os parênteses executam o getter e devolvem o valor guardado no closure. Do mesmo modo, `setNome(123)` seria rejeitado porque `T` foi inferido como `string`.

12. Mapa mental para a prova

Conceito	Pergunta-chave	Resposta curta
T	O que ele conecta?	Entradas, saídas e propriedades do mesmo uso.
any	Qual o risco?	Remove a proteção estática e perde precisão.
extends	O que garante?	Uma estrutura mínima obrigatória.
K, V	Por que dois tipos?	Chave e valor podem ter tipos independentes.
	União ou interseção?	União: uma alternativa; interseção (&): todas.
Tupla	Por que usar?	Posições e tipos fixos, como getter/setter.
Inferência	Quem define T?	O compilador, a partir dos argumentos.

Checklist final

- ☐ Consigo explicar por que any é menos seguro?
- ☐ Sei identificar o tipo inferido em cada chamada?
- ☐ Sei diferenciar união (|) de interseção (&)?
- ☐ Sei explicar o requisito mínimo de T extends Entidade?
- ☐ Sei dizer os tipos exatos de getter e setter numa tupla?

Material elaborado a partir da Atividade 2 - Tipos Genéricos (documento fornecido), com respostas reorganizadas e correções conceituais da revisão incluída no próprio arquivo.